

Week 12: Capstone Project and Final Assessment

Phase 4: Architecture & Career | Week 12 of 12

Week Overview

Week 12 is the culmination of 11 weeks of study. You will build a complete, production-quality PostgreSQL system from scratch, conduct a peer review session, and complete the final assessment. This project becomes the centerpiece of your portfolio and a concrete demonstration of everything you have learned.

Focus: Build something you're genuinely proud of. This project represents your PostgreSQL mastery.

Learning Objectives

By the end of this week, you will be able to:

- Build a production-ready database from requirements to working system.
 - Document architectural decisions with clear justifications.
 - Peer-review another engineer's database design and provide constructive feedback.
 - Present technical work to a simulated stakeholder panel.
 - Identify the gap between where you started and where you are now.
 - Articulate your PostgreSQL skills in career conversations.
-

Capstone Project Options

Choose ONE of the following. Each is designed to showcase different strengths.

Option A: Multi-Tenant SaaS Backend

Suitable for: Backend engineers and full-stack developers.

Requirements:

- Multi-tenant architecture with schema isolation or RLS isolation
- User management: roles, permissions, SSO-ready
- Subscription billing with usage metering
- Feature flags per tenant and per user
- Audit log for all data modifications
- REST API simulation (functions that return JSON)
- Performance dashboard materialized views

Deliverables:

1. Complete schema (10+ tables) with all constraints
2. Row-Level Security policies with test verification
3. 25+ queries covering all functional requirements
4. 5 PL/pgSQL functions with full error handling
5. 3 triggers (audit, constraint enforcement, cascade)
6. EXPLAIN ANALYZE for 3 complex queries with optimization notes
7. Backup and recovery procedure document

8. Architecture decision record (ADR) — why you made each design choice

Option B: Real-Time Analytics Platform

Suitable for: Data engineers and analytics engineers.

Requirements:

- Event ingestion (append-only, partitioned by day)
- User identity resolution (anonymous to authenticated)
- Funnel analysis with configurable steps
- Cohort retention matrix
- A/B testing framework
- Dashboard with 6 materialized views
- pg_cron scheduled refresh jobs
- JSONB event properties with GIN indexing

Deliverables: Same as Option A but with emphasis on:

- Partition pruning verification
 - Materialized view refresh strategy
 - Funnel and cohort query performance
-

Option C: Financial Ledger System

Suitable for: Engineers interested in fintech/banking.

Requirements:

- Double-entry bookkeeping ledger (immutable entries)
- Multi-currency account support with FX rates
- Transaction lifecycle management
- Interest calculation and monthly posting
- Fraud detection velocity checks
- Regulatory reporting queries
- Row-Level Security for customer data isolation
- Full audit trail

Deliverables: Same as Option A but with emphasis on:

- SERIALIZABLE transaction testing
 - Ledger immutability enforcement
 - Concurrent transfer fund safety verification
-

Option D: Custom Domain (Proposal)

If you have a specific domain in mind (healthcare, logistics, gaming, HR systems), propose it by Day 1 of Week 12. Your proposal must include:

- Problem statement
 - 8+ entities to model
 - 3 complex business rules to enforce
 - 2 performance challenges to address
-

Week 12 Daily Schedule

Monday — Requirements and Schema Design (90 min)

Deliverables by end of day:

- ☐ Project option selected
- ☐ Written requirements document (1 page)
- ☐ Complete list of entities and their relationships
- ☐ First draft of all CREATE TABLE statements (rough)
- ☐ Initial ER diagram drawn on paper

Key decisions to make:

- Schema isolation strategy
- Naming conventions
- Partitioning candidates
- Index candidates
- Which business rules need triggers vs. application enforcement

Tuesday — Core Schema and Data Load (90 min)

Deliverables by end of day:

- ☐ All CREATE TABLE statements with constraints finalized
- ☐ All foreign keys and CHECK constraints in place
- ☐ Sample data: at least 3 realistic data sets (100+ rows each)
- ☐ Basic CRUD queries verified working
- ☐ At least 10 indexes created with justification

Test each table:

- Insert valid data (should succeed)
- Insert invalid data (should fail with clear error)
- Insert data that violates a business rule (trigger or constraint)

Wednesday — Business Logic and Functions (90 min)

Deliverables by end of day:

- ☐ 5 PL/pgSQL functions implementing core business logic
- ☐ All functions have input validation and error handling
- ☐ Functions tested with valid, invalid, and edge case inputs
- ☐ 3 triggers implemented and verified
- ☐ UPSERT patterns where appropriate

Function quality checklist:

- Does it handle NULL inputs?
- Does it raise meaningful exceptions?
- Does it log audit events where appropriate?
- Does it handle concurrent calls safely?

Thursday — Analytics Queries and Performance (60 min)

Deliverables by end of day:

- ☐ 25+ SQL queries demonstrating all functional requirements
- ☐ At least 3 window function queries
- ☐ At least 2 recursive CTE queries
- ☐ At least 1 materialized view
- ☐ EXPLAIN ANALYZE run on 5 queries, optimization notes documented

Performance review:

- Run each major query on realistic data volume (10k+ rows minimum)
- Verify index usage with EXPLAIN ANALYZE
- Identify any full table scans and add appropriate indexes
- Check for unused indexes

Friday — Presentation and Final Assessment (90 min)

Deliverables by end of day:

- ☐ README.md with architecture overview
- ☐ All SQL scripts organized and commented
- ☐ Peer review session (swap schemas with another learner)
- ☐ Final assessment: answer 20 questions without notes (60 min)
- ☐ Self-assessment: fill out assessment rubric

Presentation format (15 minutes):

1. [2 min] Problem you solved and why you made the choices you did
2. [5 min] Walk through the schema, highlighting key design decisions
3. [5 min] Demonstrate 3 complex queries live
4. [3 min] Performance optimization you implemented (show EXPLAIN before/after)

Peer Review Guidelines

When reviewing another person's project, evaluate:

SCHEMA REVIEW CHECKLIST:

- ☐ Are all business rules enforced at the database level?
- ☐ Are naming conventions consistent?
- ☐ Is the schema in 3NF (with documented exceptions)?
- ☐ Are foreign keys defined correctly?
- ☐ Are CHECK constraints meaningful?
- ☐ Would this schema scale to 100x more data?
- ☐ Are there any obvious missing tables or columns?

QUERY REVIEW CHECKLIST:

- ☐ Do queries return correct results?
- ☐ Are edge cases handled (NULLs, empty sets)?
- ☐ Are window functions used where appropriate?
- ☐ Is EXPLAIN ANALYZE documented?
- ☐ Are there any obvious performance issues?

FEEDBACK FORMAT:

"I noticed [observation]. This could lead to [consequence]. Consider [suggestion]."
Never: "This is wrong." Always: "This could be improved by..."

Final Assessment

Format: 60 minutes, closed notes, 20 questions

Section 1: SQL Coding (20 minutes, 5 questions) Write correct SQL queries from scratch. Each query graded on:

- Correctness (does it return right results?)
- Efficiency (are obvious optimizations applied?)
- Readability (is it clean and commented?)

Section 2: Schema Design (15 minutes, 2 questions) Design a normalized schema from written requirements. No ER diagram needed — CREATE TABLE statements.

Section 3: Concept Questions (15 minutes, 8 questions) Short verbal or written answers. Target: 2 minutes per answer.

Section 4: Performance (10 minutes, 5 questions) Read an EXPLAIN ANALYZE output and answer questions about it.

Grading Rubric (see `assessment_rubric.md`)

Dimension	Weight	Description
Schema Quality	25%	Normalization, constraints, naming
Query Correctness	25%	Accuracy, edge cases, efficiency
Function/Trigger Quality	20%	Logic, error handling, concurrency safety
Performance	20%	Index strategy, EXPLAIN usage
Documentation	10%	Comments, architecture decisions, README

Portfolio Presentation

After Week 12, you have a portfolio containing:

1. 9 complete projects (01-09 in 22_Projects/)
 - Each with schema, queries, functions, triggers
2. A capstone project
 - Production-ready, documented, peer-reviewed
3. Weekly SQL challenges
 - Solved under timed conditions
4. Mock interview performance
 - Self-scored across all dimensions

GitHub Repository Structure:

```
postgresql-portfolio/
├── projects/
│   ├── 01_library/          # Schema + queries
│   ├── 02_student/
│   └── ...
├── capstone/
│   ├── schema.sql
│   ├── queries.sql
│   ├── functions.sql
│   ├── triggers.sql
│   ├── indexes.sql
│   ├── sample_data.sql
│   └── README.md           # Architecture decisions
├── interview_prep/
└── solved_challenges/
```

Self-Assessment Checklist

- ☐ My capstone schema is fully normalized (or denormalization is documented)
- ☐ All business rules are enforced at the database level
- ☐ All PL/pgSQL functions have error handling
- ☐ I ran EXPLAIN ANALYZE on every complex query
- ☐ I completed the peer review for another learner
- ☐ I took the final assessment under closed-note conditions
- ☐ I have a public GitHub repository with my work
- ☐ I can present my capstone in 15 minutes

Program Completion

Congratulations on completing the 12-week PostgreSQL Mastery Capstone Program.

You started by learning what a database is. You now:

- Understand how PostgreSQL stores, indexes, and retrieves data internally.
- Can design production database schemas for any domain.
- Write complex analytical SQL that would impress any interview panel.
- Know how to protect, monitor, and recover a production database.
- Can architect a high-availability database cluster.
- Have a portfolio of 10 complete database projects.

The skills you built in this program are durable, in-demand, and compound over time.

The only thing left is to use them.

Resources

- Final reference: [24_Cheat_Sheets/](#)
- Company prep: [23_Company_Preparation/](#)

- Ongoing practice: <https://pgexercises.com>, <https://leetcode.com/tag/database>